Arduino Design

Componente Arduino: emite el iBeacon con el tipo y el valor de la medición. Notación: Logical Design & Reverse Engineering Specification v3.

Component Design

El sketch de la plaquita (ESP32) anuncia un **iBeacon** cuyo `major` codifica el tipo de medición y el valor. Se compone de los elementos siguientes.

Elementos y responsabilidades

Elemento	Tipo	Responsabilidad
`HolaMundoIBeacon.ino`	Sketch	`setup()` / `loop()`: coordinación general
`Publicador` (`Publicador.h`)	Lógica	Calcula el `major` y controla el anuncio
`EmisoraBLE`	Comunicación	Configura y emite el anuncio iBeacon
`PuertoSerie`	E/S	Mensajes por el monitor serie

Tipos, aliases y constantes

```
MedicionesID = { CO2 = 1, TEMPERATURA = 2, PRUEBA = 3 }   // enum de Publicador

CallbackConexionEstablecida   = ( connHandle: N ) -> void
CallbackConexionTerminada     = ( connHandle: N, reason: N ) -> void
CallbackCaracteristicaEscrita = ( connHandle: N, chr: Caracteristica, data: [ N ],
len: N ) -> void

UUID        = [ N ]_16      // "EPSG-GTI-MARC-3A"
MAJOR       = ( codigo << 8 ) | contador
VALOR_MINOR = 1234
INTERVALO_EMISION   = 5000 ms
DURACION_ANUNCIO    = 1000 ms
MS_ENTRE_MEDICIONES = 2000 ms
```

Diseño de clases (notación v3)

Cada clase se dibuja con su caja: los **datos** (atributos) van dentro de la caja, y los **métodos públicos** cruzan la pared (`|`), de modo que el nombre del método queda sobre la línea (por eso esas líneas no llevan `|`). A la izquierda, `-->` apunta al método cuando recibe datos y `<--` sale de él cuando devuelve. A la derecha, `-->` indica que el método **modifica** el estado, `<--` que solo lo **consulta** y `--x` que es **estático** (no usa estado). Entre métodos consecutivos, dos líneas verticales `|`.

Sketch `HolaMundoIBeacon` (módulo de funciones libres)

Las variables que viven dentro de un grupo (`namespace`) se escriben con el prefijo del grupo: `Globales::` para los objetos globales y `Loop::` para el estado que usa el `loop()`. Las constantes de primer nivel (sin grupo) van sin prefijo.

```
      ----------------------- HolaMundoIBeacon -----------------------
      |
      |   VALOR_MINOR: Z
      |   INTERVALO_EMISION: N
      |   DURACION_ANUNCIO: N
      |   DURACION_FASES: [ N ]_8
      |   LUZ_ENCENDIDA: [ B ]_8
      |   NUM_FASES_LUZ: N
      |
      |   Globales::elLED: LED
      |   Globales::elPuerto: PuertoSerie
      |   Globales::elPublicador: Publicador
      |   Globales::elMedidor: Medidor
      |   Loop::cont: N
      |   Loop::instanteEmision: N
      |   Loop::instanteFaseLuz: N
      |   Loop::faseLuz: N
      |   Loop::anunciando: B
      |
      |
      --> setup() -->
      |
      |
      --> loop() -->
      |
      |
      --> avanzarLuces() -->
      |
      ------------------------------------------------------------------
```

Publicador

```
                  -------------------- Publicador --------------------
                  |
                  | beaconUUID: [ N ]_16
                  | laEmisora: EmisoraBLE
                  | RSSI: Z
                  |
                  |
                  --> Publicador() -->
                  |
                  |
                  --> encenderEmisora() -->
                  |
                  |
valorCO2: Z, contador: N, tiempoEspera: N --> publicarCO2() -->
```

```
                  |
                  |
valorTemperatura: Z, contador: N, tiempoEspera: N --> publicarTemperatura() -->
                  |
                  |
tipo: N, contador: N --> calcularMajor() --x
                 major: N <--
                  |
                  |
valor: Z, contador: N, tiempoEspera: N --> publicarValor() -->
                  |
                  |
                  --> pararAnuncio() -->
                  |
                  |
B <-- hayAnuncioEnCurso() <--
                  |
                  ------------------------------------------------------
```

EmisoraBLE

```
                  -------------------- EmisoraBLE --------------------
                  |
                  | nombreEmisora: Text
                  | fabricanteID: N
                  | txPower: Z
                  |
                  |
nombre: Text, fabricanteID: N, txPower: Z --> EmisoraBLE() -->
                  |
                  |
                  --> encenderEmisora() -->
                  |
                  |
cbce: CallbackConexionEstablecida, cbct: CallbackConexionTerminada
                                                --> encenderEmisora() -->
                  |
                  |
                  --> detenerAnuncio() -->
                  |
                  |
B <-- estaAnunciando() <--
                  |
                  |
uuid: [ N ]_16, major: Z, minor: Z, rssi: N --> emitirAnuncioIBeacon() -->
                                              B <--
                  |
                  |
carga: Text, tamanyoCarga: N --> emitirAnuncioIBeaconLibre() -->
                  |
                  |
```

```
servicio: ServicioEnEmisora --> anyadirServicio() -->
                                          B <--
                  |
                  |
servicio: ServicioEnEmisora --> anyadirServicioConSusCaracteristicas() -->
                                          B <--
                  |
                  |
servicio: ServicioEnEmisora, caracteristica: Caracteristica, resto...: T
                            --> anyadirServicioConSusCaracteristicas() -->
                                          B <--
                  |
                  |
servicio: ServicioEnEmisora, resto...: T
                            --> anyadirServicioConSusCaracteristicasYActivar() -->
                                          B <--
                  |
                  |
cb: CallbackConexionEstablecida --> instalarCallbackConexionEstablecida() -->
                  |
                  |
cb: CallbackConexionTerminada --> instalarCallbackConexionTerminada() -->
                  |
                  |
connHandle: N --> getConexion() <--
    Conexion <--
                  |
                  ------------------------------------------------------
```

`ServicioEnEmisora` **(con su clase interna** `Caracteristica`**)**

```
                  ------------------ ServicioEnEmisora ------------------
                  |
                  | uuidServicio: [ N ]_16
                  | elServicio: BLEService
                  | lasCaracteristicas: [ Caracteristica* ]
                  |
                  |
nombreServicio: Text --> ServicioEnEmisora() -->
                  |
                  |
                  --> escribeUUID() <--
                  |
                  |
car: Caracteristica --> anyadirCaracteristica() -->
                  |
                  |
                  --> activarServicio() -->
                  |
                  |
                  --> operator BLEService&() <--
```

```
                  |
                  ----------------------------------------------------------


                  ------------- Caracteristica (interna) -------------
                  |
                  | uuidCaracteristica: [ N ]_16
                  | laCaracteristica: BLECharacteristic
                  |
                  |
nombre: Text --> Caracteristica() -->
                  |
                  |
nombre: Text, props: N, permisoRead: SecureMode, permisoWrite: SecureMode, tam: N
                                              --> Caracteristica() -->
                  |
                  |
props: N, permisoRead: SecureMode, permisoWrite: SecureMode, tam: N
                       --> asignarPropiedadesPermisosYTamanyoDatos() -->
                  |
                  |
texto: Text --> escribirDatos() -->
      N <--
                  |
                  |
texto: Text --> notificarDatos() -->
      N <--
                  |
                  |
cb: CallbackCaracteristicaEscrita --> instalarCallbackCaracteristicaEscrita() -->
                  |
                  |
                  --> activar() -->
                  |
                  |
                  | // métodos privados internos:
                  | //   asignarPropiedades( props )                    -->
setProperties()
                  | //   asignarPermisos( permisoRead, permisoWrite ) -->
setPermission()
                  | //   asignarTamanyoDatos( tam )                    -->
setMaxLen()
                  |
                  ----------------------------------------------------------
```

PuertoSerie, **Medidor** y **LED**

```
                  -------------------- PuertoSerie --------------------
                  |
                  |
baudios: N --> PuertoSerie() -->
```



Función libre `esperar()`

```
t: N --> esperar() -->
```

Codificación del `major`

```
major = ( codigo << 8 ) | contador
        // codigo: CO2=1 | TEMPERATURA=2 | PRUEBA=3
        // contador: 0..255, se reinicia al llegar a 256
```

Design Clarifications

- El **valor** de la medición viaja en el `minor` (constante `VALOR_MINOR`, hoy `1234`); el **tipo** viaja codificado en el `major`.
- El anuncio dura `DURACION_ANUNCIO` y se repite cada `INTERVALO_EMISION`.
- El `contador` del `major` permite distinguir anuncios consecutivos del mismo tipo.
- La plaquita **no** conoce el servidor: solo emite; es la app Android quien interpreta el anuncio y lo sube.
- `publicarValor()` es el **único método nuevo** (no estaba en el esqueleto): emite un iBeacon estándar con el tipo `PRUEBA` en el `major` y el valor (`minor`) en el `minor`. Recibe `tiempoEspera` por homogeneidad con `publicarCO2()` y `publicarTemperatura()`, pero **no** espera con `delay()`: quien para el anuncio es el `loop()`.
- Los tipos y aliases (`MedicionesID`, callbacks) se declaran arriba, en *Tipos, aliases y constantes*; en las cajas solo van datos (dentro) y métodos (cruzando la pared).
- Un mismo **nombre de método repetido** en una caja son **sobrecargas** (distinta lista de parámetros), no un duplicado: en `EmisoraBLE`, `encenderEmisora()` (sin argumentos) y `encenderEmisora( cbce, cbct )`; y `anyadirServicioConSusCaracteristicas` con y sin características. La lista de entradas a la izquierda las distingue.

General Rules

- **Programming Language:** C++ (Arduino / ESP32).
- **Function/Method Headers:** cada función incluye su diseño lógico en un bloque de comentario delimitado por líneas discontinuas (`--------------------`): `entrada --> funcion() --> salida`.
- **Code Readability:** código claro y autoexplicativo; comentarios inline mínimos.
- **Automated Testing:** pruebas unitarias de la lógica pura (`test_publicador.cpp`), compilables con un compilador de C++, sin la plaquita.